

AUTONOMOUS BIO-AGENT & TOOL ORCHESTRATION

바이오 분석 에이전트 및 자동 실행 시스템

LLM 추론, 도구 스키마(Tool Schema), Function Calling 기반 자율 생물정보학 분석 파이프라인

—— 생명과학 AI 엔지니어링 실무 강의 교재

바이오 분석 에이전트 3대 핵심 설계 모듈



함수 스키마 정의

JSON 규격 도구 인터페이스.

NCBI 및 분석 함수 입출력 타입 명세화



ReAct 추론 루프

생각과 행동의 연속 반복.

다단계 생물학적 가설 수립 및 검증



격리 실행 샌드박스

파이썬 코드 안전 자율 구동.

동적 스크립트 실행 및 에러 자가 치유

Function Calling 기반 외부 바이오 시스템 연동

의도 파악과 구조화된 인자 추출

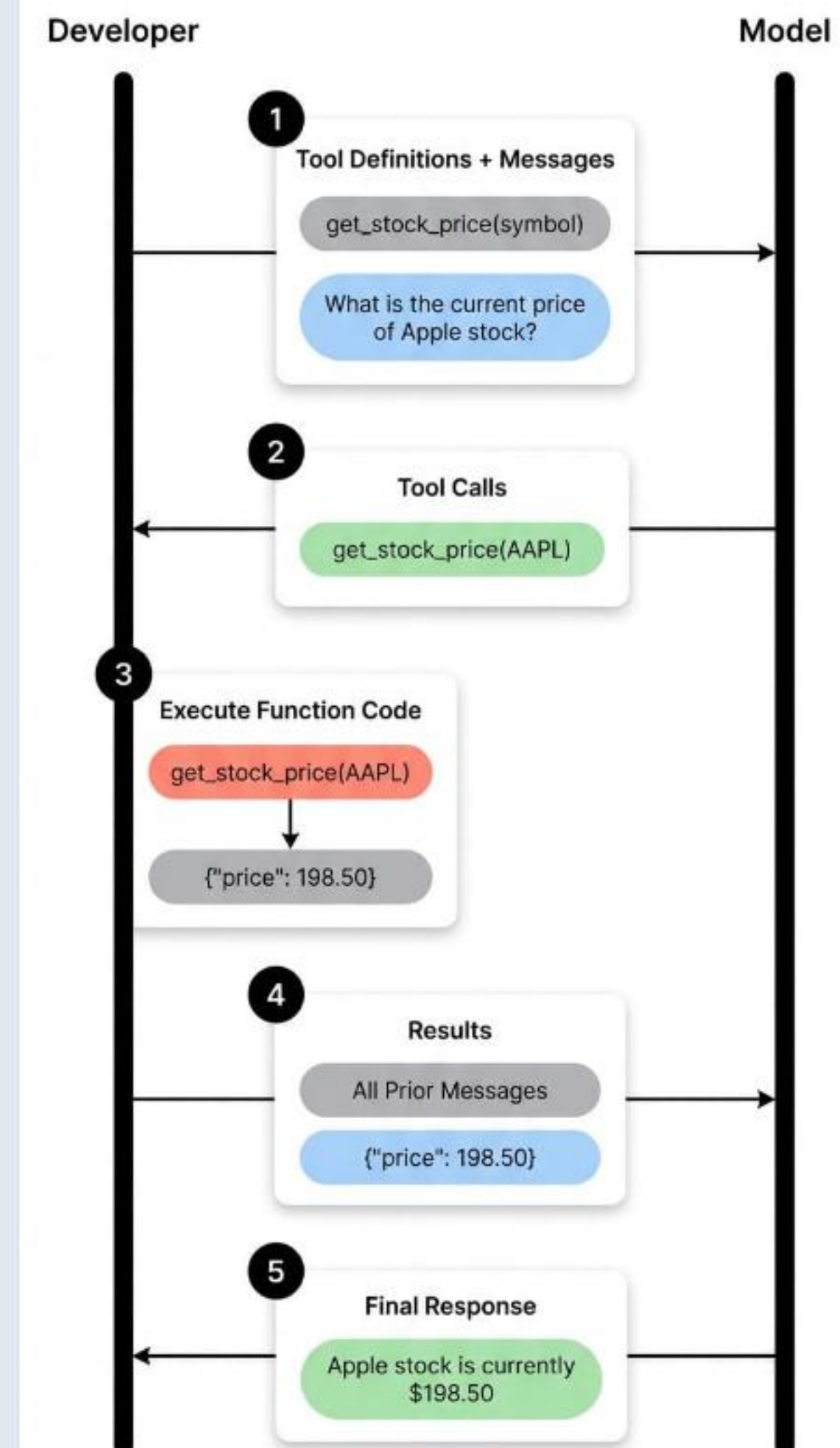
자연어 질의에서 유전자 심볼 및 분석 옵션을 JSON 매핑.

역할의 엄격한 분리

LLM은 호출 지시서를 생성하고 백엔드가 실제 함수 실행.

결과 환류와 최종 답변 통합

API 실행 결과 수치를 모델이 해석해 임상 인사이트 생성.



에이전트 연동 주요 바이오인포매틱스 툴체인



NCBI Entrez & GEO

공공 유전체 실시간 수집.

Gene ID 매핑 및 질환 코호트 메타데이터 로드



DESeq2 & Scanpy 엔진

차등 발현 유전자(DEG) 연산.

벌크 및 단일세포 전사체 자동 통계 처리



신약 화합물 탐색 (RDKit)

분자 지문 및 구조 유사도.

SMILES 기반 약물 타겟 결합 후보 스크리닝

샌드박스 기반 코드 실행 및 보안 무결성

도커 컨테이너 기반 격리 파이프라인

에이전트가 자동 작성한 파이썬 스크립트를 독립 환경에서 실행.

환자 개인정보(PHI) 유출 방지

원시 식별 번호를 마스킹 처리하여 안전한 통계 요약치만 모델에 전달.

실행 타임아웃 및 자원 쿼터

초고용량 게놈 연산 무한 루프 차단 및 메모리 초과 자동 격리.

샌드박스 보안 통제 규칙

외부 네트워크 아웃바운드	화이트리스트만 허용
파일 시스템 권한	임시 디렉토리 읽기/쓰기
실행 시간 제한	단일 잡당 180초
입출력 데이터 감사	전체 Trace 로그 저장

전통적 하드코딩 파이프라인 vs 바이오 에이전트

전통적 고정형 스크립트 방식

경직된 파이프라인: 입력 데이터 서식 변경 시 실행 중단.

수동 에러 디버깅: 라이브러리 충돌 시 연구자가 직접 수정.

상황별 분기 한계: 새로운 생물학적 가설마다 코드 재작성.

자율형 바이오 에이전트 시스템

동적 도구 선택: 질의 의도에 최적화된 API 자율 호출.

자율 에러 치유: Traceback 분석 후 매개변수 즉시 보정.

다학제 연계 해석: 통계 지표와 최신 학술 문헌 자동 결합.

오류 자가 수정(Self-Correction) 및 반추 메커니즘



런타임 예외 해석

파이썬 에러 로그 파싱.

차원 불일치(Shape Mismatch) 원인 자동 식별



인자 재구성 및 재시도

매개변수 수정 루프.

적절한 임계치(Threshold)로 쿼리 자동 재호출

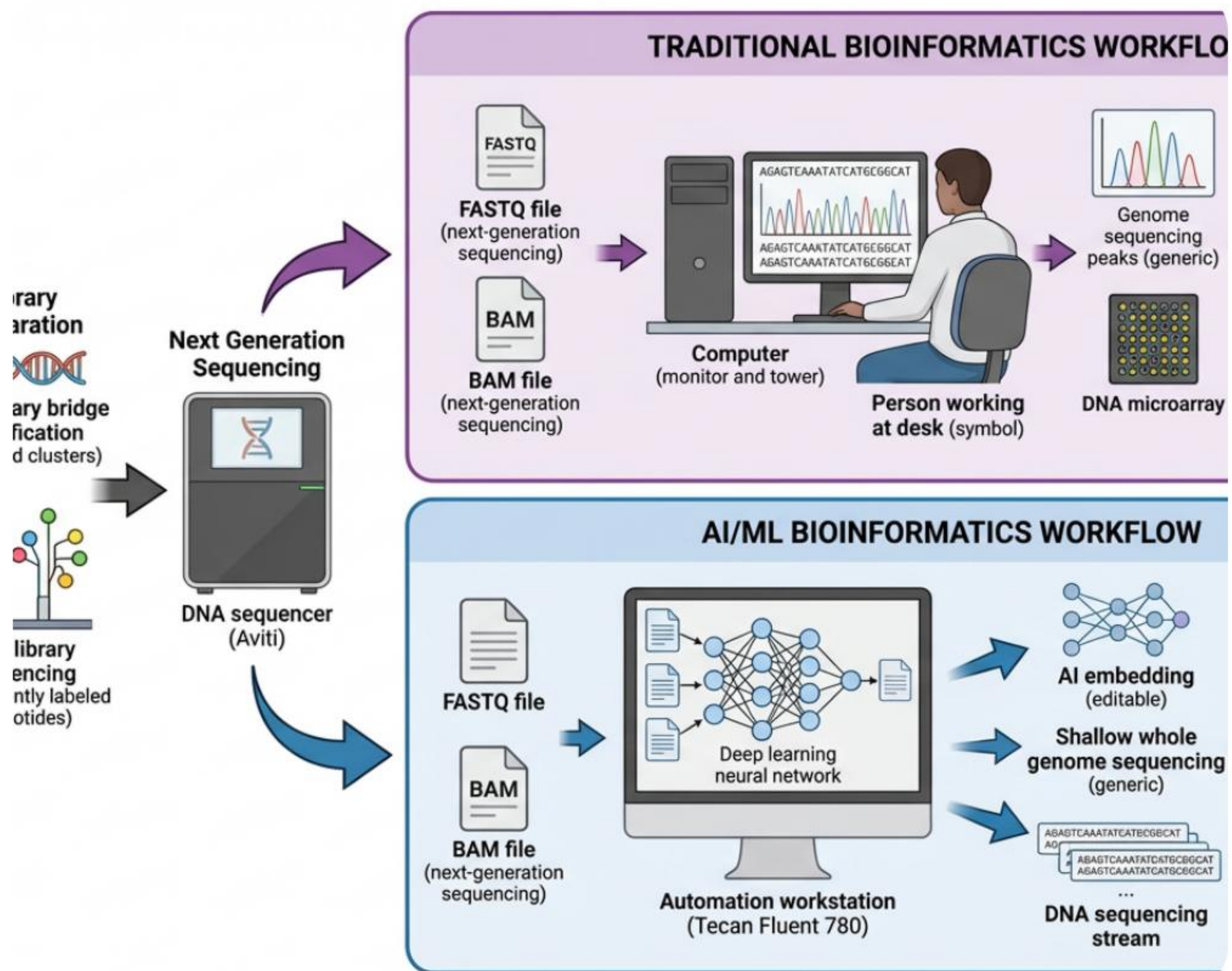


생물학적 타당성 검증

지표 범위 상식 검사.

발현 배수(Fold Change) 음수 발생 여부 교차 검증

역할 분담 기반 멀티 에이전트 협업 체계



플래너(Planner): 분석 계획 수립

임상 연구 목적에 따라 전처리-DEG-경로 분석 순서 결정.

코더(Coder): 스크립트 작성 및 도구 구동

Bioconductor 패키지 호출 및 파이썬 데이터 조작 코드 실행.

리뷰어(Reviewer): 통계 유효성 평가

FDR 보정 여부와 생물학적 결론의 신뢰도를 최종 검수.

자연어 질의부터 자율 분석 완료까지 4단계

01

자연어 질의 입력

연구자 의도 파악.
특정 암종 바이오마커 질의

02

도구 스키마 매핑

Function Calling 생성.
JSON 규격 파라미터 빌드

03

샌드박스 자율 실행

생물정보학 연산 수행.
에러 감지 시 자율 디버깅

04

임상 리포트 합성

시각화 및 요약 완성.
연구자 의사결정 보조

바이오 분석 에이전트 도입에 따른 효율성 제고

전통적 연구원 수작업 스크립트 작성 및 파이프라인 연계 시간

8.0 시간

Function Calling 에이전트 자율 분석 시스템 소요 시간

0.9 시간 (-88.7%)

8.8×

분석 속도 향상 배율

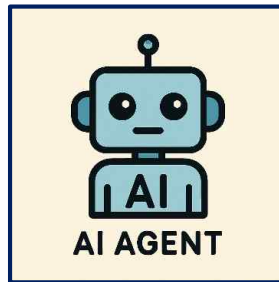
97.4%

함수 호출 스키마 적합도

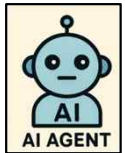
92.1%

런타임 에러 자율 치유율

AI 에이전트와 랭체인 소개



1.1 AI 에이전트 소개



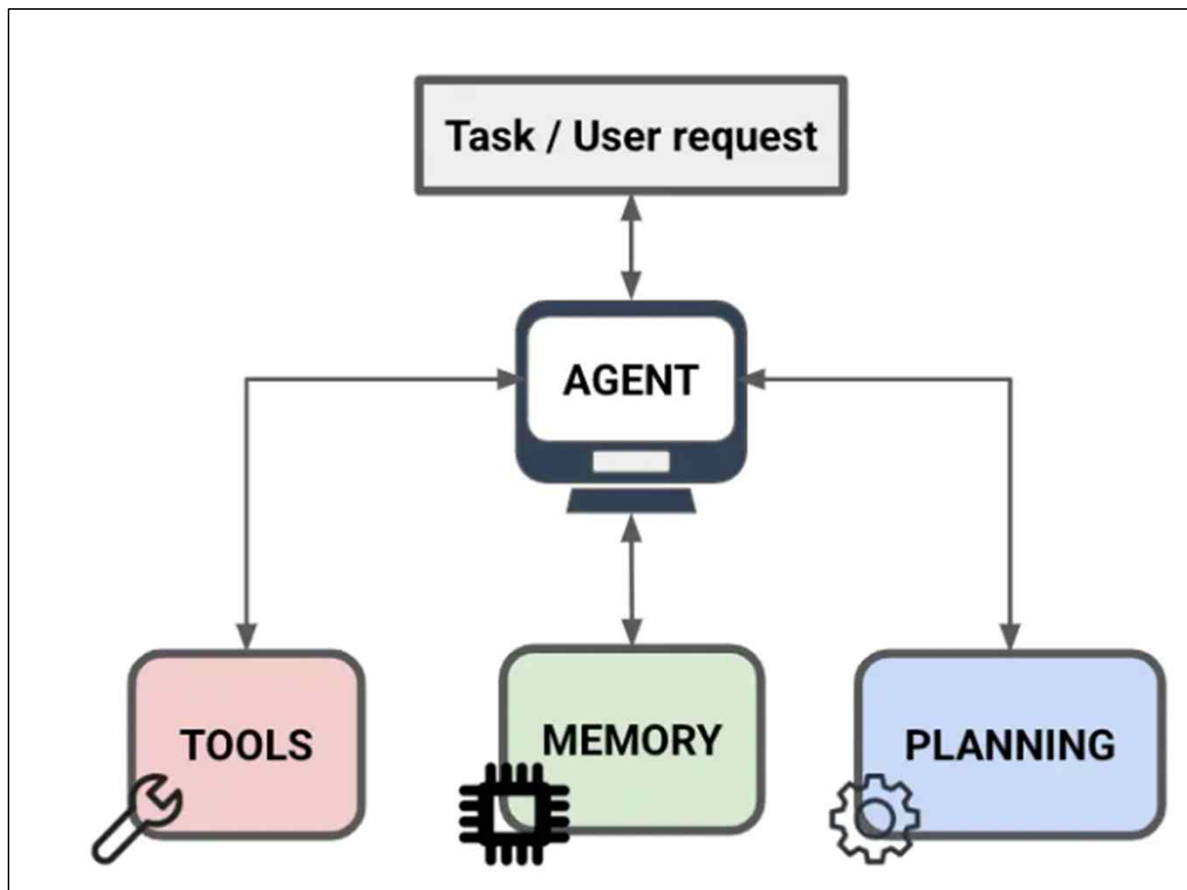
AI 에이전트란?

AI 에이전트는 사용자의 요청이나 환경의 변화를 인식하고, 이를 기반으로 **목표 달성을 위해 자율적으로 판단·행동하는 인공지능 시스템**입니다.

LLM(대규모 언어 모델)을 기반으로 하며, 단순 질의응답(Q&A)을 넘어 **툴 사용, 데이터 검색, 외부 API 호출, 문서 처리** 등 복합적인 작업을 자동화할 수 있습니다.

1.1 AI 에이전트 소개

AI 에이전트의 구성 요소



<https://www.promptingguide.ai/agents/introduction>

AI 에이전트의 구성 요소별 기능

1. Task / User Request (사용자 요청)

- 사용자가 에이전트에게 수행을 요청하는 작업
예: "오늘 날씨 알려줘", "이 문서 요약해줘", "데이터 분석해줘"
 - 이 요청이 에이전트의 입력이 됩니다.
-

2. Agent (에이전트 본체)

- 중앙 컨트롤러 역할을 하는 인공지능 모듈
- 사용자 요청을 분석하고, 필요한 리소스(도구·메모리·계획)를 조합해 작업을 수행
- LLM(대규모 언어모델) 기반으로 동작하며, 상황에 따라 외부 API나 내부 데이터베이스를 호출

1.1 AI 에이전트 소개

3. TOOLS (도구)

- 에이전트가 작업을 수행하기 위해 호출할 수 있는 기능 모듈
 - 예:
 - 웹 검색 API
 - 계산기
 - 데이터베이스 쿼리
 - 파일 읽기/쓰기
 - LangChain에서는 "Tool" 객체로 정의해 에이전트가 선택적으로 호출
-

4. MEMORY (메모리)

- 에이전트가 이전 대화·작업 이력을 기억하는 저장소
- 단기 메모리: 현재 세션의 문맥 유지
- 장기 메모리: 여러 세션에 걸쳐 지속적으로 기억
- 예:
 - 사용자의 이름/취향 저장
 - 지난 회의 내용 기억
- 이를 통해 연속적이고 개인화된 대화 가능

1.1 AI 에이전트 소개

5. PLANNING (계획 수립)

- 복잡한 작업을 수행할 때, 어떤 순서로 어떤 도구와 데이터를 사용할지 결정
- 예:
 1. 사용자 요청 분석
 2. 필요한 데이터 수집(툴 호출)
 3. 결과 가공
 4. 최종 답변 생성
- "Task decomposition(작업 분해)"과 "Reasoning(추론)" 과정이 포함

동작 흐름

1. 사용자 요청이 들어옴
2. **Agent**가 요청을 해석
3. 필요 시 **계획(Planning)** 단계에서 처리 순서 결정
4. 필요한 ****도구(Tools)****나 ****메모리(Memory)****를 활용
5. 최종 결과를 사용자에게 응답

1.1 AI 에이전트 소개

AI 에이전트는 **인식, 처리, 행동, 학습 및 적용, 자율성**의 5가지 주요 특징을 바탕으로 수행 및 문제 해결

특징	내용
인식(Perception)	• 센서, 데이터 파일, 인터넷 등 다양한 데이터 수집 방법을 통해 주변 환경 인식 및 파악
처리(Brain)	• 입력 데이터 기반 주요 정보 추출 및 새로운 정보 학습을 통한 의사 결정
행동(Action)	• 도출된 결정, 해답 기반 텍스트 생성, API 호출, 물리적 동작 등 실제 수행
학습 및 적용 (Learning and Adaptation)	• 경험 기반 지속적 학습, 과거 데이터를 기반으로 미래 행동 개선
자율성(Autonomy)	• 스스로 결정을 내리고 행동하며 인간의 개입 없이 목표를 달성하기 위한 작업 수행

* 자료 : SK C&C(2024)

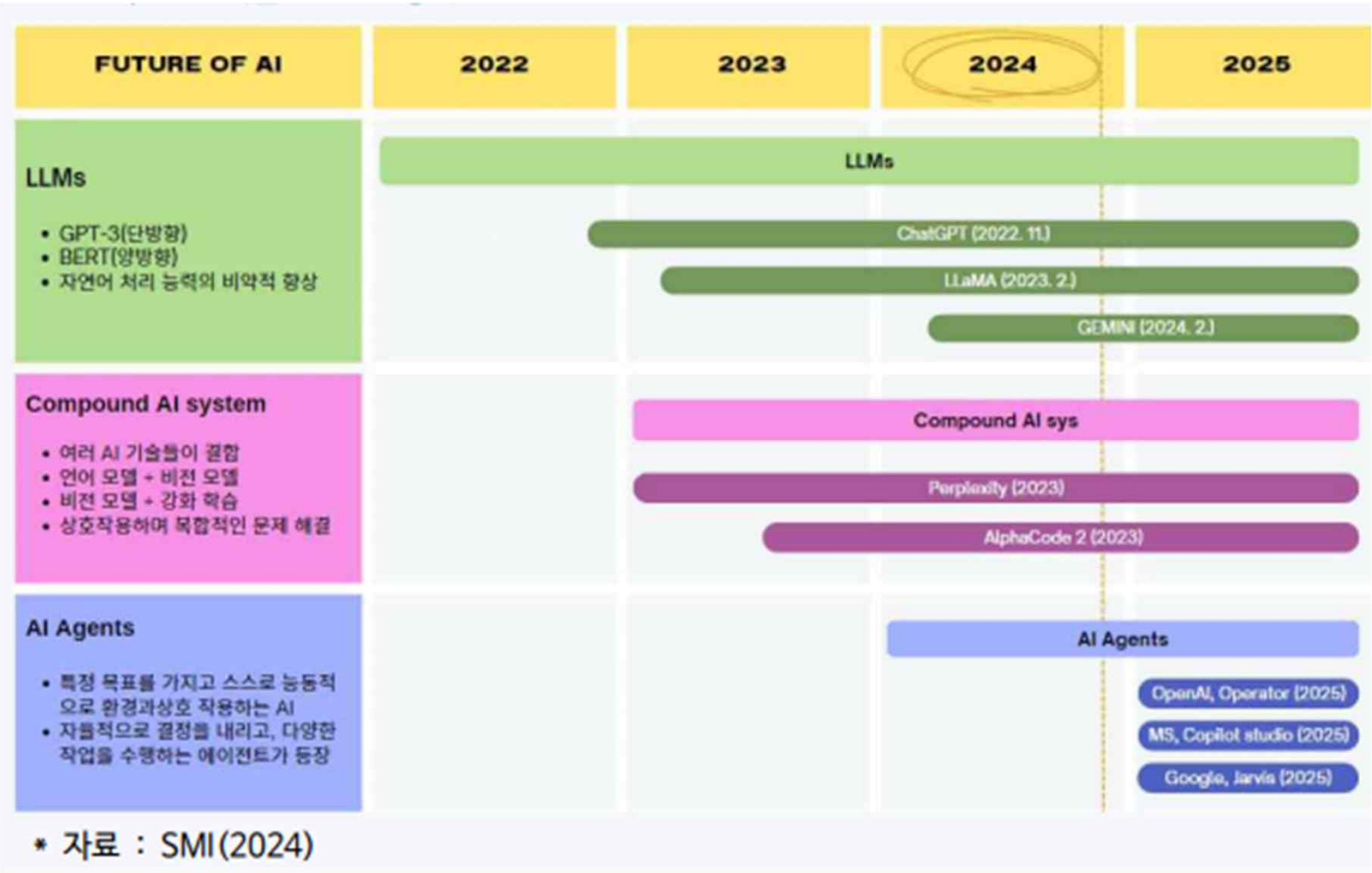
1.1 AI 에이전트 소개

	 일반적인 언어 모델	 AI 에이전트
사용 범위	프로세스 자동화	전체 워크플로우 및 프로세스 자동화
계획	워크플로우를 계획하거나 조율할 수 없는 경우	사용자의 목표를 달성하기 위해 다단계 계획을 수립 및 실행하여, 실시간 피드백에 따라 작업 조정
메모리 및 미세조정	메모리 유지 및 미세 조정 기능이 제한적	단기 및 장기 메모리를 활용하여 이전 사용자 상호 작용에서 학습하고 개인화된 응답을 제공하며, 시스템 내 여러 에이전트 간에 메모리 공유 가능
도구 통합	본질적으로 외부 도구 또는 시스템과 통합되도록 설계되지 않음	API와 도구(데이터 추출기, 이미지 선택기, 검색API)로 내재된 언어 모델의 기능을 강화하여 작업 수행
데이터 통합	고정된 학습 데이터와 훈련 시점에 기반한 정적 지식에 의존	새로운 정보와 실시간 지식 소스에 동적으로 적응
정확성	일반적으로 자체 평가 기능이 부족하고 학습 데이터에 기반한 확률적 추론으로 제한	특정 작업에 특화된 기능, 지식, 메모리를 활용하여 시스템 내 자체 출력물과 다른 에이전트의 출력물을 검증하고 개선 가능

* 자료 : Deloitte(2024) 재구성

1.1 AI 에이전트 소개

AI 기술 발전 현황과 주요 모델 , AI 에이전트



AI 에이전트의 일반적인 사용 사례

업계에서 에이전트가 적용되는 일반적인 사용 사례의 전체 목록은 다음과 같습니다.

- 추천 시스템: 제품, 서비스 또는 콘텐츠에 대한 제안을 개인화합니다.
- 고객 지원 시스템: 문의 처리, 문제 해결, 지원 제공.
- 연구: 법률, 금융, 건강 등 다양한 분야에 걸쳐 심층적인 조사를 수행합니다.
- 전자상거래 애플리케이션: 온라인 쇼핑 경험 향상, 주문 관리, 개인화된 추천 제공.
- 예약: 여행 준비 및 이벤트 계획 지원.
- 보고: 방대한 양의 데이터를 분석하고 포괄적인 보고서를 생성합니다.
- 재무 분석: 시장 동향을 분석하고, 재무 데이터를 평가하고, 전례 없는 속도와 정확성으로 보고서를 생성합니다.

<https://www.promptingguide.ai/agents/introduction>

1.2 랭체인 소개

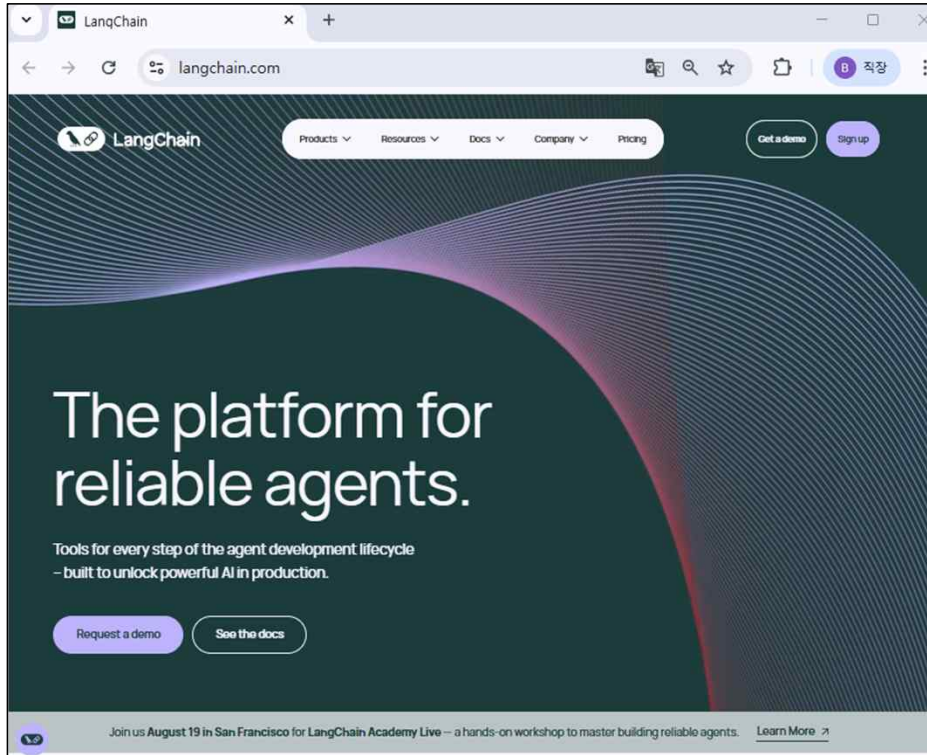


1.2 랭체인 소개

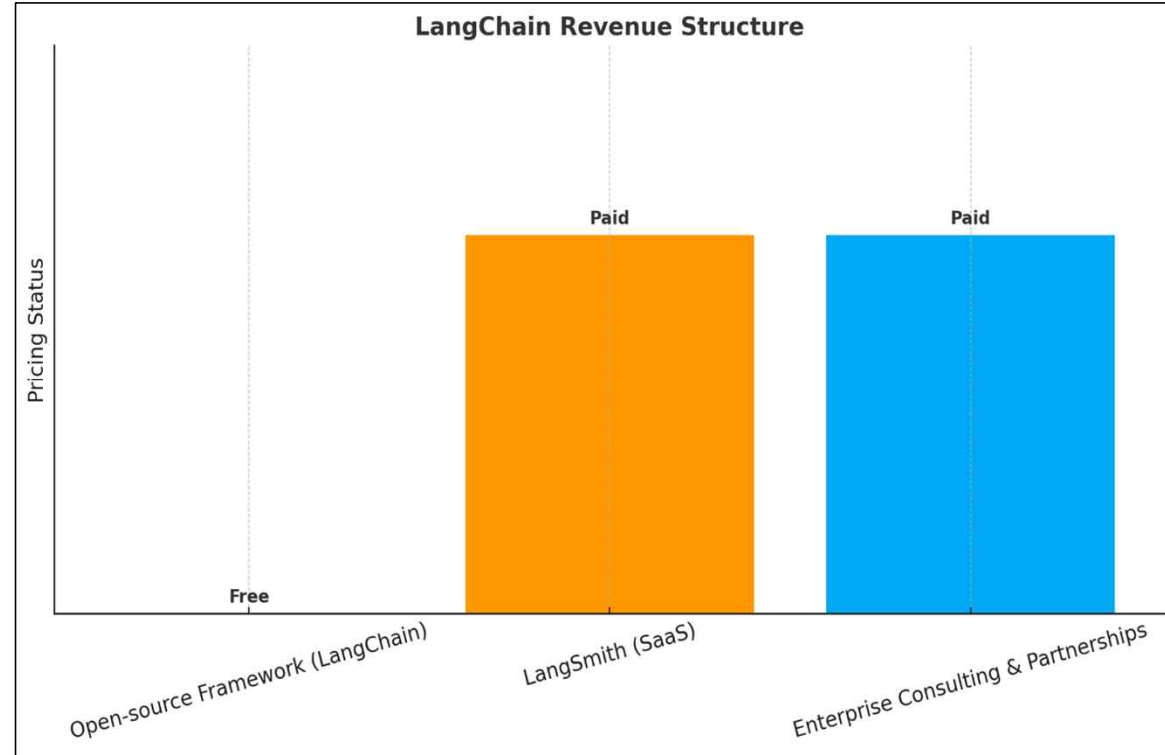
랭체인이란?

- 랭체인(LangChain): 언어 모델 기반 AI 애플리케이션을 쉽게 개발할 수 있도록 돕는 프레임워크.
- 장점
 - 검색, 문서 생성, 질의응답 등 기능을 손쉽게 구현 가능
 - 복잡한 애플리케이션 개발 시 미리 준비된 모듈을 활용해 개발 속도 향상
- 특징
 - 다양한 언어 모델(Gemini, Llama, Claude, GPT 등) 간 손쉬운 교체 가능
 - 코드 변경 없이 언어 모델을 바꿔 쓸 수 있어 특정 모델의 장점을 활용 가능
- 활용 효과
 - 특정 모델에 종속되지 않고 유연하고 확장성 있는 AI 애플리케이션 개발 가능

1.2 랭체인 소개



<https://www.langchain.com/>



랭체인의 수익 구조

1.2 랭체인 소개

1 프로젝트 시작 배경

- 2022년 10월, 해리슨 체이스(Harrison Chase)가 오픈 소스 프로젝트로 시작
- Robust Intelligence에서 근무하면서 **LLM 애플리케이션 개발에 필요한 공통 패턴과 도구의 부재**를 느끼고,
이를 해결하기 위해 누구나 쉽게 챗봇, 질의응답, 요약 시스템을 만들 수 있는 프레임워크 구상

2 주요 목적과 특징

- **LLM 애플리케이션과 파이프라인을 신속하게 구축**할 수 있는 플랫폼 제공
- 프롬프트 관리, 체인(Chain) 구성, 외부 데이터 연결(RAG), 메모리·에이전트 기능까지
LLM 활용에 필요한 모든 구성요소를 모듈화해 개발자가 재사용 가능하도록 설계

3 성장 과정

- 2023년 4월, 법인 전환 및 세쿼이아 캐피탈 등 VC 투자 유치 → 커뮤니티와 생태계 빠르게 성장
- 오픈소스 커뮤니티 중심으로 다양한 통합(데이터베이스, 벡터DB, API 등) 생태계 확장

4 최신 동향

- 2024년 1월, **v0.1.0 첫 안정(stable) 버전** 공개
- 그 전까지 지적받던 잦은 업데이트·호환성 문제·버그를 대폭 개선
- 보다 일관된 **API**와 개발자 친화적 문서 제공으로 프로덕션에서도 활용하기 쉬워짐

1.2 랭체인 소개

랭체인이란?

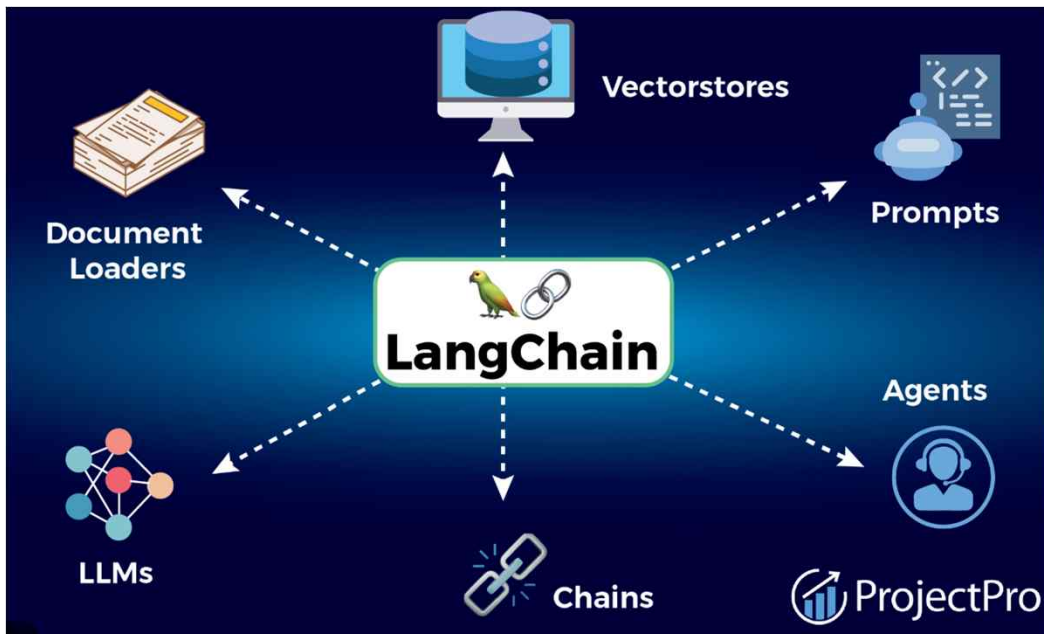
- 랭체인은 챗봇, Q&A 시스템 등 **LLM 기반 애플리케이션**을 쉽게 만들 수 있도록 돕는 프레임워크입니다.
- LLM 애플리케이션 개발 시 필요한 다양한 도구를 연결하고 활용하는 역할을 합니다.
- 예:
 - **Document Loader**로 문서를 불러올 수 있음
 - `WebBaseLoader` → 웹 페이지 내용을 불러오기
 - `PDFLoader` → PDF 내용 불러오기
 - `Text Splitter` → 긴 텍스트를 분할하여 처리

즉, 랭체인은 **LLM 애플리케이션 제작에 필요한 데이터 로딩·처리 도구들을 쉽게 연계할 수 있게** 해주는 도구입니다.

1.2 랭체인 소개

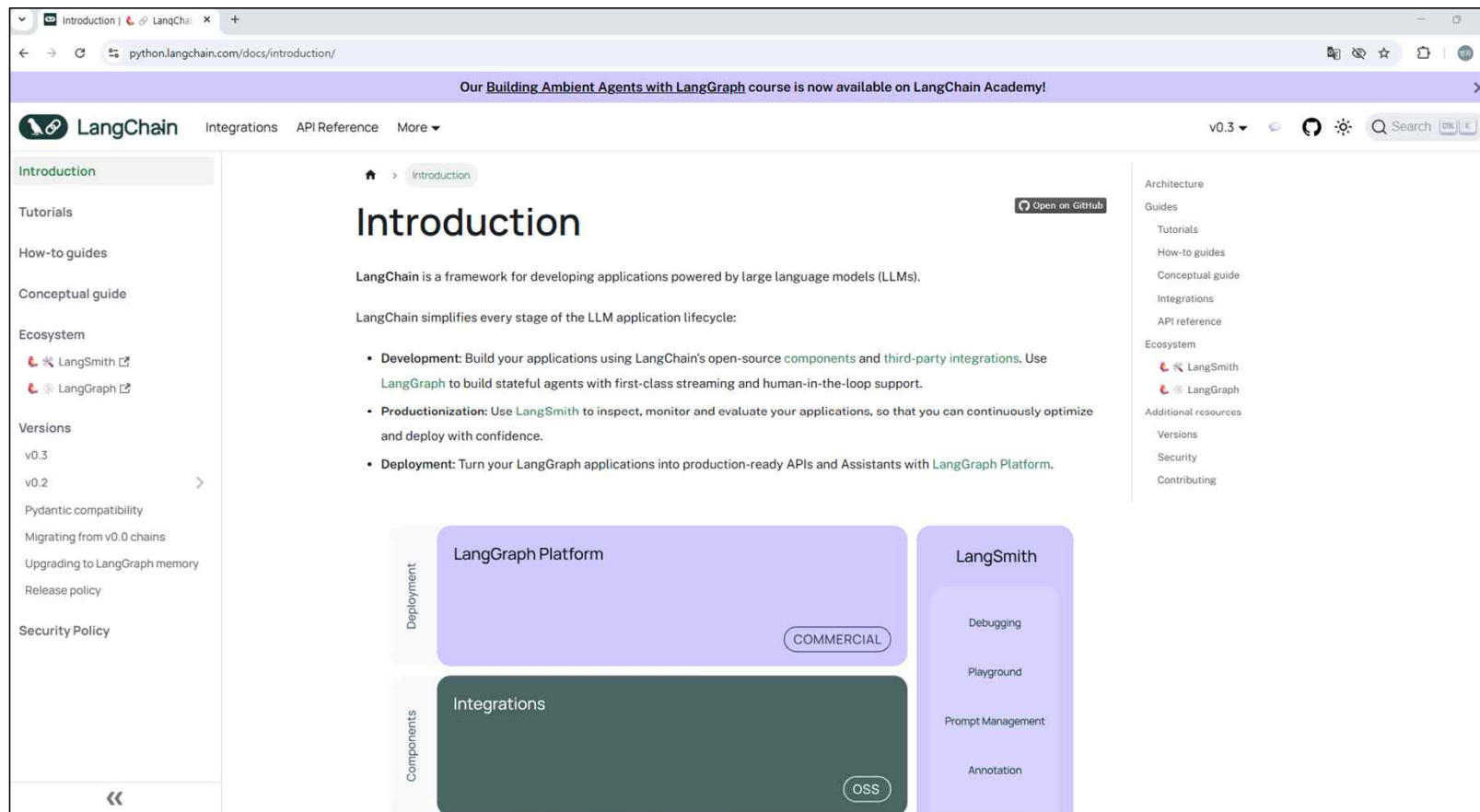
🌐 LangChain 구성 요소

- **Language** → 언어 모델(LLM, Large Language Model)을 의미
- **Chain** → 여러 기능(프롬프트, 데이터 로더, 파서 등)을 연결(**Chaining**) 하여 파이프라인처럼 동작



1.2 랭체인 소개

랭체인 개발자를 위한 공식 사이트 : <https://python.langchain.com/>



소개

[Open on GitHub](#)

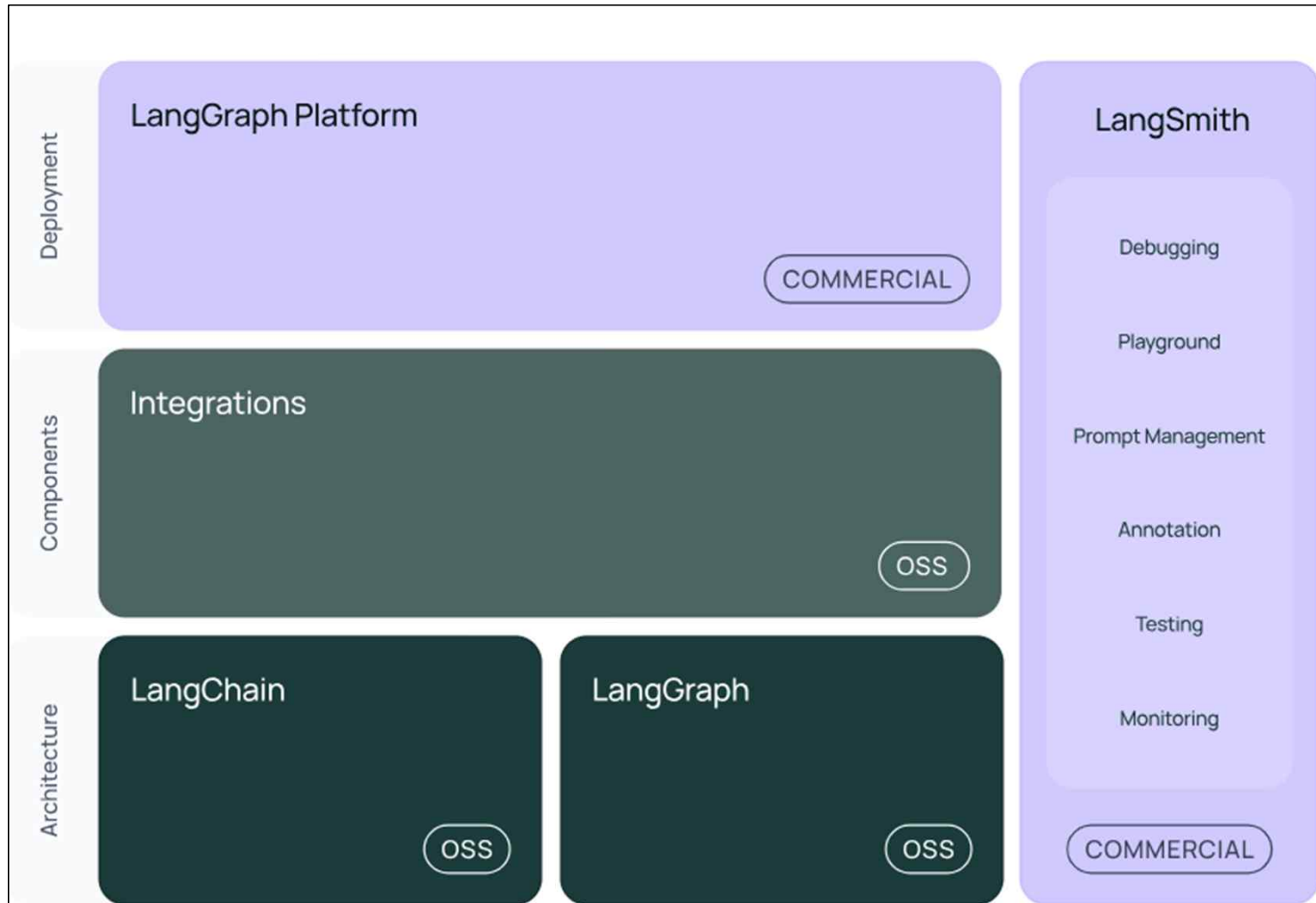
LangChain은 대규모 언어 모델(LLM)을 기반으로 애플리케이션을 개발하기 위한 프레임워크입니다.

LangChain은 LLM 애플리케이션 라이프사이클의 모든 단계를 단순화합니다.

- **개발** : LangChain의 오픈 소스 구성 요소와 타사 통합을 활용하여 애플리케이션을 구축하세요. LangGraph를 사용하여 최고 수준의 스트리밍 및 HIL(Human-in-the-Loop) 지원을 제공하는 상태 저장 에이전트를 구축하세요.
- **생산화** : LangSmith를 사용하여 애플리케이션을 검사, 모니터링 및 평가함으로써 지속적으로 최적화하고 자신감을 가지고 배포할 수 있습니다.
- **배포** : LangGraph 플랫폼을 사용하여 LangGraph 애플리케이션을 프로덕션에 바로 적용 가능한 API와 Assistant로 전환하세요.

<https://python.langchain.com/docs/introduction/>

1.2 랭체인 소개

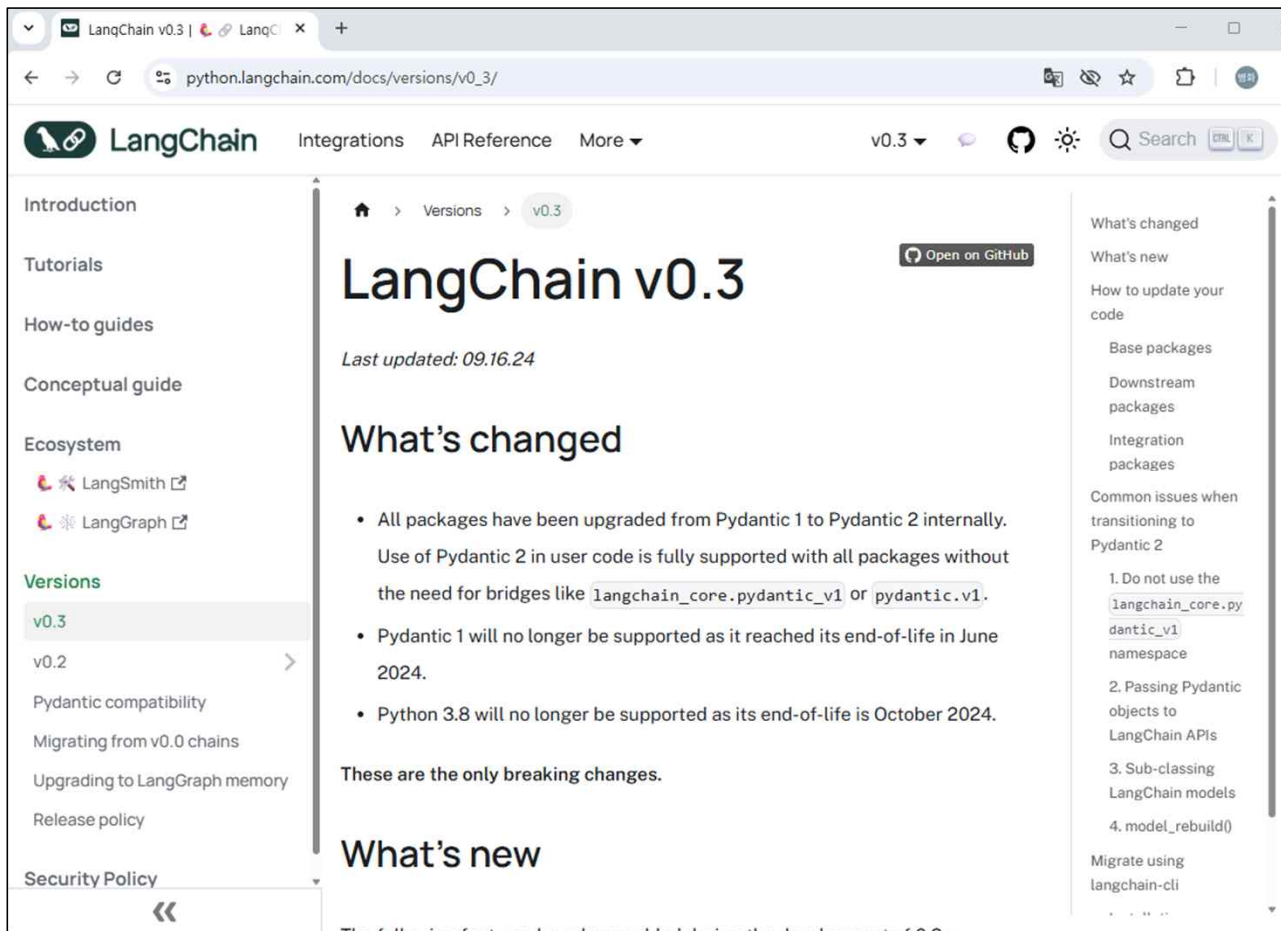


<https://python.langchain.com/docs/introduction/>

1.3 랭체인 환경 설정

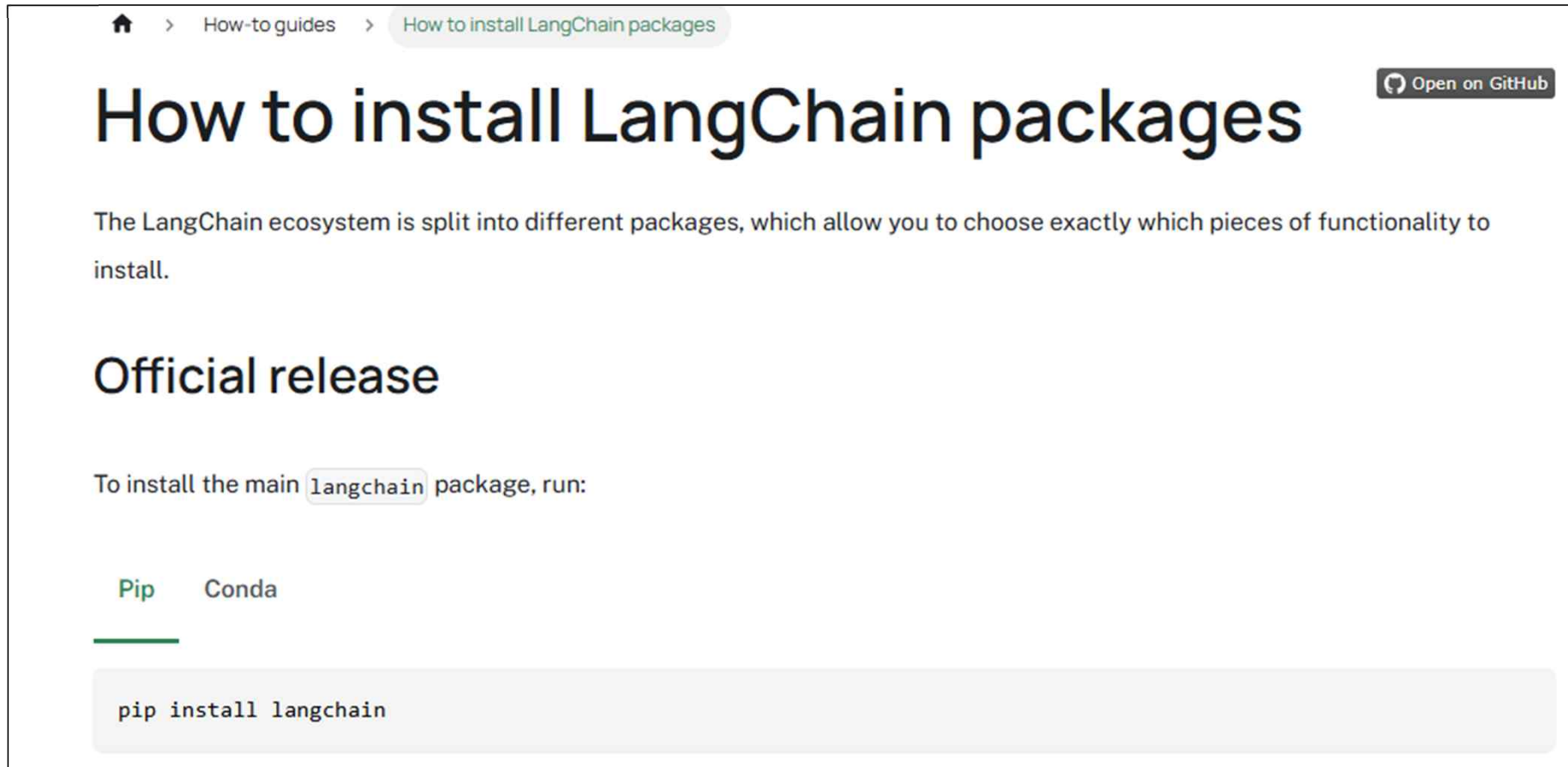
1.3 랭체인 환경 설정

랭체인 v0.3 공식 문서 : https://python.langchain.com/docs/versions/v0_3/



1.3 랭체인 환경 설정

 랭체인 패키지 설치 : **`pip install langchain`**



The screenshot shows the 'How to install LangChain packages' page from the official documentation. It includes a breadcrumb trail, a title, a brief introduction, and instructions for installing the main package using pip or conda. The 'Pip' section is highlighted with a green underline, and the command `pip install langchain` is shown in a code block.

Home > How-to guides > How to install LangChain packages

How to install LangChain packages

[Open on GitHub](#)

The LangChain ecosystem is split into different packages, which allow you to choose exactly which pieces of functionality to install.

Official release

To install the main `langchain` package, run:

Pip Conda

```
pip install langchain
```

https://python.langchain.com/docs/how_to/installation/

1.3 랭체인 환경 설정

pip install langchain

- langchain-core (기본 로직)
- 문서 로더(Document Loaders)
- 벡터 스토어(Vector Stores)
- 에이전트/툴 연결(Agents & Tools)
- 통합 API 클라이언트(OpenAI, HuggingFace, Pinecone 등)

pip install langchain-core

- **LangChain의 핵심 로직만 설치**
- 프롬프트 관리, 콜백, 실행 체인 같은 **코어 기능만 제공**
- 벡터DB, 데이터 로더, 외부 API 커넥터 등은 **포함되지 않음**
- 가볍게 필요한 기능만 쓰거나, 맞춤형 최소 환경을 구성할 때 사용

명령어	포함 범위	용도
pip install langchain	코어 + 확장 모듈 + 통합 라이브러리	대부분의 개발자, 전체 기능 사용
pip install langchain-core	코어 기능만	경량 설치, 맞춤형 환경 구성

1.3 랭체인 환경 설정

Jupyter Notebook 에서 LangChain 패키지 설치 : **%pip install langchain langchain-openai**

```
[1]: %pip install langchain langchain-openai
```

```
Collecting langchain
  Downloading langchain-0.3.27-py3-none-any.whl.metadata (7.8 kB)
Collecting langchain-openai
  Downloading langchain_openai-0.3.30-py3-none-any.whl.metadata (2.4 kB)
Collecting langchain-core<1.0.0,>=0.3.72 (from langchain)
  Downloading langchain_core-0.3.74-py3-none-any.whl.metadata (5.8 kB)
Collecting langchain-text-splitters<1.0.0,>=0.3.9 (from langchain)
  Downloading langchain_text_splitters-0.3.9-py3-none-any.whl.metadata (1.9 kB)
Collecting langsmith>=0.1.17 (from langchain)
  Downloading langsmith-0.4.14-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in c:\users\storm\anaconda3\lib\site-packages (from langchain) (2.8.2)
Requirement already satisfied: SQLAlchemy<3,>=1.4 in c:\users\storm\anaconda3\lib\site-packages (from langchain) (2.0.34)
Requirement already satisfied: requests<3,>=2 in c:\users\storm\anaconda3\lib\site-packages (from langchain) (2.32.3)
Requirement already satisfied: PyYAML>=5.3 in c:\users\storm\anaconda3\lib\site-packages (from langchain) (6.0.1)
Collecting openai<2.0.0,>=1.99.9 (from langchain-openai)
  Downloading openai-1.99.9-py3-none-any.whl.metadata (29 kB)
Collecting tiktoken<1,>=0.7 (from langchain-openai)
  Downloading tiktoken-0.11.0-cp312-cp312-win_amd64.whl.metadata (6.9 kB)
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in c:\users\storm\anaconda3\lib\site-packages (from langchain-openai) (8.2.3)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in c:\users\storm\anaconda3\lib\site-packages (from langchain-core<1.0.0,>=0.3.72) (1.33)
Requirement already satisfied: typing-extensions>=4.7 in c:\users\storm\anaconda3\lib\site-packages (from langchain-core<1.0.0,>=0.3.72) (4.7.1)
```

랭체인 기본



1.1 LangChain Expression Language (LCEL)

1.1 LangChain Expression Language (LCEL)



🌸 LangChain Expression Language (LCEL)란?

LCEL은 LangChain에서 체인(**Chain**)과 **Runnable**을 조합해서 파이프라인을 만드는 표준 문법이에요.

쉽게 말해, LangChain의 여러 구성 요소(PromptTemplate, Model, Parser 등)를 ****|** (파이프 연산자)******로 연결해서 하나의 흐름으로 실행할 수 있게 만든 표현식 언어입니다.

1.1 LangChain Expression Language (LCEL)

🔑 핵심 특징

1 파이프라인 스타일 구성

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Translate this to Korean: {text}")
model = ChatOpenAI(model="gpt-4o-mini")
parser = StrOutputParser()

chain = prompt | model | parser
chain.invoke({"text": "Hello, how are you?"})
```

- `|` (파이프 연산자)로 연결하면 입력 → 프롬프트 → 모델 → 파서가 순서대로 실행
- 함수형 프로그래밍 스타일이라 직관적이고 테스트·조합이 쉬움

1.1 LangChain Expression Language (LCEL)

2 모든 구성 요소가 Runnable

- PromptTemplate, Model, Retriever, Parser 등은 모두 Runnable 인터페이스 구현
- .invoke(), .batch(), .stream() 같은 공통 메서드로 실행 가능 → 동기/비동기/스트리밍 쉽게 전환

3 체인 컴포저빌리티(조합 가능성)

- 여러 개의 체인을 묶어 더 큰 체인으로 만들 수 있음
- 입력·출력을 map/dict 형태로 변환하여 병렬 실행도 가능

```
from langchain_core.runnables import RunnableParallel

chain1 = prompt1 | model | parser
chain2 = prompt2 | model | parser

multi_chain = RunnableParallel(first=chain1, second=chain2)
```

1.1 LangChain Expression Language (LCEL)

프로덕션 친화적

- 일관된 인터페이스 덕분에 테스트·캐싱·로깅·모니터링이 쉬움
- 최근 LangChain은 기존 `LLMChain` 대신 LCEL 방식을 적극 권장

장점 정리

장점	설명
간결한 코드	여러 단계를 한 줄로 연결해 직관적
재사용성	개별 Runnable 블록 재사용 가능
유연한 실행	<code>.invoke()</code> , <code>.stream()</code> , <code>.batch()</code> 모두 지원
조합성	파이프라인, 병렬 실행, 분기처리까지 쉽게 구성
프로덕션 준비	로깅/관측성/캐싱/병렬 처리 내장

1.1 LangChain Expression Language (LCEL)

요약

LCEL은 LangChain의 새로운 표준 방식으로,

"프롬프트 → 모델 → 파서" 같은 데이터 흐름을 간단하게 표현하고 실행하는 문법

이 덕분에 코드를 깔끔하게 유지하면서도
테스트, 확장, 프로덕션 배포가 쉬워집니다.

1.2 LCEL 인터페이스

(Runnable 프로토콜)

1.2 LCEL 인터페이스

사용자 정의 체인을 가능한 쉽게 만들 수 있도록, Runnable 프로토콜을 제공한다. Runnable 프로토콜은 대부분의 컴포넌트에 구현되어 있다. 이는 표준 인터페이스로, 사용자 정의 체인을 정의하고 표준 방식으로 호출하는 것을 쉽게 만듭니다. 표준 인터페이스에는 다음이 포함됩니다.

- [stream](#): 응답의 청크를 스트리밍합니다.
- [invoke](#): 입력에 대해 체인을 호출합니다.
- [batch](#): 입력 목록에 대해 체인을 호출합니다.

1.2 LCEL 인터페이스

- 비동기 메소드

[astream](#) : 비동기적으로 응답의 청크를 스트리밍합니다.

[ainvoke](#) : 비동기적으로 입력에 대해 체인을 호출합니다.

[abatch](#) : 비동기적으로 입력 목록에 대해 체인을 호출합니다.

[astream_log](#) : 최종 응답 뿐만 아니라 발생하는 중간 단계를 스트리밍합니다.

1.2 LCEL 인터페이스

stream: 실시간 출력

`chain.stream` 메서드는 실시간 토큰 스트리밍을 지원합니다.

모델이 응답을 생성할 때 토큰이 만들어지는 즉시 전달되고,
이를 반복(iterate)하며 출력하면 타이핑하듯 한 글자씩 결과가 나타납니다.

- `end=""` → 출력 시 줄바꿈 없이 이어서 표시
- `flush=True` → 출력 버퍼를 바로 비워 즉시 화면에 반영

예시 코드

```
# chain.stream 메서드를 사용하여 "멀티모달" 주제에 대한 스트림 생성
for token in chain.stream({"topic": "멀티모달"}):
    # 토큰을 줄바꿈 없이 이어서 출력하고, 즉시 화면에 반영
    print(token, end="", flush=True)
```

1.2 LCEL 인터페이스

invoke: 호출

`chain.invoke()` 메서드는 한 번의 입력을 받아 해당 입력에 대해 체인을 실행하고 결과를 반환합니다.

- 입력은 보통 딕셔너리 형태(`dict`)로 전달합니다.
- 키는 프롬프트에서 사용하는 변수명, 값은 그 변수에 들어갈 실제 데이터입니다.

예시 코드

```
# chain 객체의 invoke 메서드를 호출하고,  
# 'ChatGPT' 라는 값을 topic 변수에 전달하여 실행  
chain.invoke({"topic": "ChatGPT"})
```

➡ 실행 결과는 모델의 응답(출력 텍스트 또는 파싱된 결과)이 리턴됩니다.

1.2 LCEL 인터페이스

특징

- 단일 입력만 처리 (한 번 호출 = 한 번 응답)
- 동기 방식 실행 → 함수 호출하듯 결과를 바로 얻음
- 여러 개의 입력을 처리하려면 `.batch()` 또는 `.stream()` 을 사용

1.2 LCEL 인터페이스

batch: 배치(단위 실행)

`chain.batch()` 메서드는 여러 입력을 한 번에 처리할 때 사용합니다.

- 입력으로 ****딕셔너리들의 리스트(list)****를 받습니다.
- 각 딕셔너리의 키는 프롬프트 변수명, 값은 해당 변수에 넣을 데이터입니다.
- 모든 입력을 순차 또는 병렬로 실행한 뒤 결과 리스트를 반환합니다.

예시 코드

```
# 여러 개의 topic 값을 한 번에 처리
chain.batch([
    {"topic": "ChatGPT"},
    {"topic": "Instagram"}
])
```

➡ 두 입력이 각각 실행되고, 결과는 리스트로 반환됩니다.

`results[0]`, `results[1]` 형태로 각각 확인할 수 있습니다.

1.2 LCEL 인터페이스

Parallel: 병렬 실행

LangChain Expression Language(LCEL)는 **병렬 요청**을 지원합니다.
여러 개의 체인을 동시에 실행하고, 결과를 묶어서 반환할 수 있습니다.
이를 위해 `RunnableParallel` 클래스를 사용합니다.

예시 코드

```
from langchain_core.runnables import RunnableParallel
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI(model="gpt-4o-mini")
```

1.2 LCEL 인터페이스

```
# 1 {country}의 수도를 물어보는 체인
chain1 = (
    PromptTemplate.from_template("{country}의 수도는 어디야?")
    | model
    | StrOutputParser()
)

# 2 {country}의 면적을 물어보는 체인
chain2 = (
    PromptTemplate.from_template("{country}의 면적은 얼마야?")
    | model
    | StrOutputParser()
)

# 3 두 체인을 병렬로 실행하는 RunnableParallel 객체 생성
combined = RunnableParallel(capital=chain1, area=chain2)

# 4 호출: 두 체인이 동시에 실행되고 결과는 딕셔너리 형태로 반환
result = combined.invoke({"country": "한국"})
print(result)

# 출력 예시: {"capital": "서울", "area": "약 100,210 km²"}
```

1.2 LCEL 인터페이스

특징

- 여러 체인을 동시에 실행하므로 속도가 빠름 (병렬 처리)
- 결과는 지정한 키(capital, area 등)로 묶인 딕셔너리 형태로 반환
- 하나의 입력을 받아 서로 다른 질문을 동시에 던지고, 응답을 합쳐서 활용 가능

1.3 랭체인 메시지 클래스

1.3 랭체인 메시지 클래스

◆ 주요 메시지 클래스

LangChain에서는 다음과 같은 메시지 클래스가 제공된다.

- `SystemMessage`
 - 대화 전반에 대한 규칙, 지침을 담는다.
 - 예: `SystemMessage(content="너는 친절한 수학 선생님이다.")`
- `HumanMessage`
 - 사용자가 입력한 메시지를 나타낸다.
 - 예: `HumanMessage(content="2+2는 얼마인가?")`
- `AIMessage`
 - 모델이 출력한 응답을 나타낸다.
 - 예: `AIMessage(content="답은 4이다.")`
- `FunctionMessage`
 - 도구 호출 또는 함수 실행 결과를 담는다.
 - 예: API 호출 결과를 전달할 때 사용된다.

1.3 랭체인 메시지 클래스

◆ HumanMessage가 필요한 이유

1. 멀티턴 대화 지원

- 문자열 입력만으로는 대화가 길어질수록 역할 구분이 어렵다.
- `HumanMessage`, `AIMessage` 를 사용하면 대화 히스토리를 리스트 형태로 관리할 수 있다.

2. 역할(Role) 명확화

- 모델은 문장이 사람의 질문인지, 시스템 지침인지에 따라 다르게 반응한다.
- 예: `SystemMessage` 에서 "짧게 답하라"라고 지정하면 모든 답변이 짧게 나온다.

3. 에이전트 및 툴 연동 확장성

- 추후 Agent 구현 시 `HumanMessage`, `AIMessage`, `FunctionMessage`를 조합하여 툴 호출이나 계산기 사용 같은 구조적 대화를 구현할 수 있다.

1.3 랭체인 메시지 클래스

예제 소스

```
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

messages = [
    SystemMessage(content="너는 친절한 조교야."),
    HumanMessage(content="안녕?"),
    AIMessage(content="안녕하세요! 무엇을 도와드릴까요?"),
    HumanMessage(content="LangChain이 뭐야?")
]

response = model.invoke(messages)
print(response.content)
```

1.3 랭체인 메시지 클래스

한줄 요약

단순 문자열은 "한 번 질문하고 끝"에 적합하고,

`HumanMessage` 방식은 대화 히스토리 유지 + 역할 지정 + 메타데이터 + 함수 호출까지 가능해서
실제 서비스나 프로젝트 만들 때 훨씬 유리하다.

1.4 멀티 턴(Muti-turn)

1.4 멀티 턴(Multi-turn)

● 멀티 턴(Multi-Turn) 대화란?

멀티 턴 대화는 사용자와 모델이 여러 차례 주고받으며 맥락(context)을 유지한 대화를 말합니다. 즉, 한 번의 질문(턴) + 답변(턴)으로 끝나는 싱글 턴(single-turn) 대화와 달리, 이전 대화 내용을 기억하고 그 위에 계속 질문/답변을 이어가는 방식입니다.

💬 예시

✅ 싱글 턴

User: 파리의 수도는 어디야?

AI: 프랑스의 수도는 파리입니다.

- 질문 → 답변 → 끝
- 이전 대화를 기억하지 않음

1.4 멀티 턴(Multi-turn)

✓ 멀티 턴

User: 파리의 인구는?

AI: 약 2,100,000명입니다.

User: 거기서 가장 유명한 관광지는?

AI: 에펠탑, 루브르 박물관, 노트르담 대성당 등이 있습니다.

- 두 번째 질문은 *****파리*****라는 이전 맥락을 활용
- AI가 문맥을 유지해서 적절한 답변 가능

1.4 멀티 턴(Multi-turn)



멀티 턴의 핵심 요소

요소	설명
대화 히스토리 저장	이전 사용자 질문과 모델 답변을 함께 기억
맥락 유지	새 질문을 해도 이전 대화와 연결 지어 해석
지시/역할 지속	초반에 설정한 시스템 프롬프트(예: "친절한 과학 선생님")가 계속 유지

1.4 멀티 턴(Multi-turn)

● 멀티 턴 대화 구현 + 실행 예시

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# 1 모델 생성
model = ChatOpenAI(model="gpt-4o-mini")

# 2 세션별 대화 기록 저장소
store = {}

# 3 세션 ID에 따라 대화 기록 가져오기
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# 4 모델에 대화 기록 기능 추가
with_message_history = RunnableWithMessageHistory(model, get_session_history)
```

1.4 멀티 턴(Multi-turn)

```
# 5 세션 ID 설정
config = {"configurable": {"session_id": "chat-001"}}

# 6 첫 번째 대화
response = with_message_history.invoke(
    [HumanMessage(content="안녕? 나는 홍길동이야!")],
    config=config,
)
print(response.content)

# 7 두 번째 대화 (이전 대화 맥락을 활용)
response = with_message_history.invoke(
    [HumanMessage(content="내 이름이 뭐지?")],
    config=config,
)
print(response.content)
```

실행 결과 예시

안녕하세요, 홍길동님! 만나서 반가워요.
당신의 이름은 홍길동입니다.

1.4 멀티 턴(Multi-turn)

🔑 작동 원리

1. 세션 관리

- `store` 라는 딕셔너리에 `session_id` 를 키로 저장
- 세션마다 별도의 대화 기록이 유지 → 여러 사용자가 동시에 접속해도 대화가 섞이지 않음

2. 대화 히스토리 저장

- `InMemoryChatMessageHistory()` 객체가 이전 메시지(Human/AI)를 메모리에 저장
- 모델 호출 시 자동으로 히스토리를 붙여서 맥락 유지

3. 실행 시 기록 포함

- `RunnableWithMessageHistory` 는 모델 실행 시 해당 세션의 대화 기록을 함께 전달
- 다음 질문을 할 때 이전 대화 내용이 자동으로 입력에 포함 → 멀티 턴 대화 가능

감사합니다